

This is the definitive blueprint for **LockedFI**. Use the first section as your project documentation/README and the second section as your personal "Design Narrative" to explain your engineering process to your mentors.

Part 1: Project Context & System Architecture

Project Name: LockedFI

Tagline: Privacy-Preserving ZK-Income Attestation & AI-Governed Smart Vaults.

Abstract:

LockedFI is a next-generation decentralized finance (DeFi) security solution that bridges the gap between off-chain identity/financial status and on-chain execution. It utilizes Zero-Knowledge Proofs (ZK-Email/DKIM) to verify a user's income threshold without exposing private data. Furthermore, it implements an AI-driven "Guardian" layer—an autonomous off-chain co-signer that performs real-time risk analysis on transactions. The system is anchored by ERC-4337 Smart Vaults, ensuring that high-value transfers require dual-signature authorization (Owner + AI).

1. Technical Stack

- **Zero-Knowledge Layer:** Circom (Circuit Compiler), SnarkJS (Groth16 Proving System).
- **Smart Contracts:** Solidity 0.8.19+, Foundry (Development Framework).
- **Account Abstraction:** ERC-4337 (Smart Accounts), custom validation logic.
- **AI Guardian Engine:** Groq Cloud (Llama-3.3-70B model), Python 3.14+, Web3.py.
- **Frontend:** Next.js 15 (App Router), TypeScript, Tailwind CSS, Ethers.js v6.
- **Infrastructure:** Fedora Linux, Anvil (Local Node), Arbitrum Sepolia (Deployment Target).

2. System Design & Architecture Layers

- **Layer 1: Identity & Privacy (ZK-Layer):** Uses DKIM signatures from emails to generate a ZK-Proof. This proof is verified on-chain to grant a "High Income Badge" without revealing the user's actual salary or email address.
- **Layer 2: Autonomous Risk Engine (AI Layer):** An off-chain service that monitors the Smart Vault. It receives transaction requests, analyzes them against user history and ZK-status, and generates a cryptographic co-signature only if the risk is deemed acceptable.
- **Layer 3: Programmable Execution (Contract Layer):** A Smart Vault that replaces traditional EOAs (Externally Owned Accounts). It features a "Dual-Control" lock: single signatures for low-risk txs, and mandatory AI co-signing for high-risk txs.
- **Layer 4: User Interface (Dashboard):** A unified interface for proof generation, vault management, and real-time transaction monitoring.

3. Project Structure

```
LockedFI/  
├── zk-artifacts/      # Compiled ZK-Circuits, .zkey, and .wasm files  
├── contracts/         # Foundry Project  
│   ├── src/          # SmartVault.sol, Verifier.sol (ZK-Verifier)  
│   ├── test/         # Integration & Unit Tests  
│   └── deploy_lockedfi.sh # Automated Deployment Orchestrator  
├── ai-guardian/       # Python AI Co-signer Module  
│   ├── guardian_service.py # Risk Analysis & Signing Logic  
│   └── .env           # AI and Guardian Private Keys  
└── frontend/         # Next.js Dashboard  
    ├── src/app/api/   # Secure API Bridge to AI Guardian  
    └── src/constants/ # Auto-synced Contract ABIs and Addresses
```

Part 2: Technical Specification: LockedFI Smart Vault

Version: 1.0.0

Status: Prototype/Research

Date: March 8, 2026

Subject: AI-Augmented Decentralized Asset Management with Zero-Knowledge Verification

1. Executive Summary

LockedFI is a decentralized asset management protocol designed to mitigate risk in high-value transactions without compromising user privacy. By integrating Account Abstraction (ERC-4337), Large Language Model (LLM) reasoning, and ZK-SNARKs, LockedFI provides a "Proof-of-Safety" layer for the Ethereum ecosystem.

2. Problem Statement

Traditional hardware wallets provide "cold" security but lack contextual awareness. Smart contract wallets offer programmability but often require complex, manual rule-setting. LockedFI addresses these gaps by introducing:

- Contextual Security:** AI-driven transaction analysis.
- Privacy-Preserving Compliance:** ZK-verified financial status.
- Programmable Risk Mitigants:** Automated threshold enforcement.

3. System Architecture

3.1 On-Chain Layer (Ethereum/EVM)

The core of the system is the `SmartVault.sol` contract, which acts as a programmable account.

- **Access Control:** Implements a multi-signature validation logic where `signature_length == 130` (65 bytes Owner + 65 bytes AI).
- **Verification Engine:** Interfaces with `IVerifier` to validate Groth16 proofs.
- **Risk Engine:** Enforces a `RISK_LIMIT` (default 0.1 ETH). Transactions exceeding this limit trigger mandatory ZK-Badge checks and AI co-signing.

3.2 Intelligence Layer (AI Guardian)

A Python-based middleware service utilizing the Groq inference engine.

- **Model:** Llama-3.3-70b-versatile.
- **Objective:** Analyze the semantic intent of a transaction (e.g., "Withdraw 5 ETH to unknown address") against the user's historical patterns and badge status.
- **Action:** Outputs a deterministic `APPROVE` or `REJECT` decision followed by a cryptographic signature on success.

3.3 Privacy Layer (Zero-Knowledge)

A ZK-SNARK circuit built with Circom.

- **Public Inputs:** Verification keys and result status.
- **Private Inputs:** User's raw financial data (income/balance).
- **Result:** Generates a proof that the user meets the "High-Income" threshold without disclosing the actual balance.

4. Operational Workflows

4.1 Transaction Lifecycle

1. **Initialization:** User requests a transaction via the Frontend.
2. **Local Signing:** User's private key signs the `UserOperation` hash.
3. **AI Interception:** The Next.js API route forwards the hash and context to the AI Guardian.
4. **Contextual Analysis:** AI evaluates risk and co-signs if safe.
5. **On-Chain Validation:** `SmartVault` verifies both signatures and checks the `hasHighIncomeBadge` mapping.
6. **Execution:** If all conditions pass, the transaction is executed.

5. Security Model

- **Trust Assumption:** The AI Guardian is a trusted co-signer in this prototype. Future iterations will explore Decentralized AI (DeAI) or TEE-based (Trusted Execution Environment) signing.
- **Encryption:** All communication between the frontend and the AI service is conducted via HTTPS.

- **Failure Modes:** If the AI service is offline, the vault defaults to a restricted state where only low-value transactions are permitted.

6. Technical Requirements

- **Runtime:** Node.js v20.x, Python 3.11+.
- **Blockchain:** EVM-compatible chain (Anvil for local development).
- **Tooling:** Foundry (Forge/Cast), SnarkJS, Circom.

7. Roadmap

- **Phase 1:** Prototype development (Current).
 - **Phase 2:** Multi-agent consensus for the Guardian layer.
 - **Phase 3:** Dynamic risk limits based on on-chain volatility.
 - **Phase 4:** Mainnet-ready security audit and KMS integration.
-

Part 3: Engineering Journey (From Scratch to Product-Ready)

This is the narrative you should use when mentors ask, "How did you build this?"

Phase 1: Cryptographic Forging (The ZK-System)

The project began by defining the privacy requirements. I designed a Zero-Knowledge circuit using **Circom** to verify that a private input (income) is greater than a public threshold. I implemented the **Groth16** proving system, performing a trusted setup (Powers of Tau) to generate the Proving and Verification keys. This allows the vault to trust a user's financial status without the user ever uploading a bank statement to the blockchain.

Phase 2: Smart Vault Engineering (The Solidity Layer)

Using **Foundry**, I developed a programmable vault based on the **ERC-4337** standard. Unlike a standard wallet, this vault contains a custom `validateUserOp` function. I integrated the ZK-Verifier directly into the contract logic. I engineered a "Dynamic Risk Engine" into the code: any transaction exceeding 0.1 ETH is automatically flagged as "High Risk," requiring both the ZK-Badge to be present and a dual-signature from an authorized guardian.

Phase 3: The AI Guardian (The Intelligence Layer)

To solve the problem of private key theft, I built an off-chain **AI Guardian**. This module acts as a digital co-signer. I developed a Python-based service that interfaces with high-performance LLMs to analyze transaction context. If the AI deems a transaction safe, it uses its own private key to generate a second cryptographic signature. I then built a **Bundler** to merge the Owner's signature and the AI's signature into a single 130-byte payload that the Smart Vault can verify.

Phase 4: Full-Stack Integration & Optimization

The final phase involved building a **Next.js** dashboard. I developed a secure API bridge that allows the frontend to communicate with the AI Guardian. To ensure the project was "Product-Ready," I wrote an automated deployment orchestrator (Bash) that handles the complex process of deploying the ZK-Verifier, initializing the Smart Vault, and synchronizing the ABIs with the frontend.

Phase 5: Testing & Security Hardening

I performed rigorous integration testing using **Anvil** and **Forge**. I simulated various attack vectors, such as unauthorized withdrawal attempts and high-risk transfers without ZK-Badges. The system successfully blocked all unauthorized actions, proving the efficacy of the dual-signature architecture.

This is your **Final Mission Briefing**. I have structured the PPT content to highlight your engineering rigor and prepared a technical "Cheat Sheet" for your Viva to ensure you can defend the architecture like a pro.

Part 4: Viva Technical Prep (Algorithm Defense)

When the mentors ask, "**Why did you use this?**" or "**How does this work?**", use these technical answers:

1. The ZK Algorithm: Groth16 (SNARKs)

- **Where:** Used in the `Verifier.sol` and `income_verifier.circom`.
- **Why:** It produces very small proofs (roughly 200 bytes) and is extremely fast to verify on-chain. This saves gas costs for the user.
- **Technical Detail:** It uses a "Trusted Setup" to create the proving and verification keys.

2. The Hashing Algorithm: Poseidon

- **Where:** Used inside the ZK-Circuit to hash the email secret.
- **Why:** Standard hashes like SHA-256 are very "expensive" (computationally heavy) inside a ZK-proof. Poseidon is a "ZK-friendly" hash designed specifically to require fewer mathematical constraints, making proof generation faster on 8GB RAM systems.

3. The Signature Algorithm: ECDSA (secp256k1)

- **Where:** Used for the Owner's signature and the AI Guardian's signature.

- **Why:** This is the native Elliptic Curve Digital Signature Algorithm used by Ethereum. By using ECDSA, our Smart Vault can use the `ecrecover` function to verify who signed the transaction without needing extra libraries.

4. The Intelligence Logic: LLM Reasoning (Llama-3 via Groq)

- **Where:** In the `guardian_service.py` off-chain module.
- **Why:** Traditional "If/Else" code is too rigid for security. An LLM can analyze *context* (e.g., "The user has a ZK-Badge and is withdrawing a slightly higher amount than usual, but it matches their income level"). This provides "Adaptive Security."

5. The Protocol: ERC-4337 (Account Abstraction)

- **Where:** The `SmartVault.sol` contract.
- **Why:** Standard Ethereum accounts (EOAs) cannot have logic. ERC-4337 allows us to turn a wallet into a smart contract. This is what enables the vault to "check" for the AI signature before it allows the money to move.

6. Why did you use Groth16 for the ZK-Proofs?

- **Answer:** Groth16 is the most efficient proving system for on-chain verification. While it requires a "Trusted Setup," the resulting proofs are very small (about 200 bytes) and the gas cost to verify them on the blockchain is the lowest among all current ZK-algorithms. This is critical for DeFi scalability.

7. Explain the Poseidon Hash function and why it's in your circuit.

- **Answer:** Standard hash functions like SHA-256 are "unfriendly" to ZK-circuits because they require millions of mathematical constraints. Poseidon is a sponge-based hash designed specifically for ZK-cryptography. It reduces the number of constraints by 10x, allowing us to generate proofs quickly even on hardware with limited RAM (like an 8GB system).

8. How does the AI Guardian actually sign a transaction?

- **Answer:** The AI itself doesn't "know" blockchain code. The Python service acts as the translator. It feeds the transaction context to the Llama-3 model. If the model outputs "APPROVE," the Python service uses the `eth_account` library to sign the transaction hash using the Guardian's private key. This signature is then sent back to the frontend to be bundled.

9. What is the security risk of your AI Guardian?

- **Answer:** Since this is a 2-of-2 model, the AI Guardian is a "Co-signer," not a "Sole-signer." If the AI is compromised, the hacker still cannot move funds because they lack

the User's primary private key. If the User's key is stolen, the AI blocks the transaction because it doesn't match the user's risk profile. It provides defense-in-depth.

10. How does the Smart Vault distinguish between a single and a dual signature?

- **Answer:** Inside the Solidity code, we check the length of the `signature` parameter. A standard signature is 65 bytes. Our bundled signature is 130 bytes. We use assembly-level "slicing" to split the 130 bytes into two 65-byte signatures and verify them against the `owner` and `aiGuardian` addresses respectively.

11. Why use ERC-4337 instead of a simple Multi-sig?

- **Answer:** ERC-4337 (Account Abstraction) allows for "Session Keys" and "Social Recovery" in the future. It makes the wallet a smart contract, which means we can change the security rules (like the 0.1 ETH risk limit) without the user needing to move their funds to a new wallet.

Final Tips for the Viva:

- **If they ask about "Scalability":** Say, *"We chose Arbitrum/L2 for deployment to ensure that the multi-signature verification remains cheap and fast for the end user."*
- **If they ask about "Security":** Say, *"The AI does not hold the user's keys. It holds its own Guardian key. Even if the AI is hacked, the hacker still needs the user's primary key to move money. It is a true 2-of-2 security model."*

Presentation Slides (1 Page per Slide)

Slide 1: Abstract

- **Title:** LockedFI: Privacy-First DeFi with AI-Governed Smart Vaults.
- **Content:** LockedFI is a decentralized financial security layer that combines Zero-Knowledge (ZK) cryptography with Artificial Intelligence. It solves the tension between financial transparency and personal privacy. By using ZK-Email proofs to verify income and an AI-driven Guardian to co-sign transactions, LockedFI creates a "Dual-Control" safe that protects assets even if a user's primary private key is compromised.

Slide 2: Introduction

- **The Problem:**
 1. **Privacy Leakage:** Proving income for DeFi loans usually requires exposing sensitive bank statements or email data on-chain.

2. **Security Fragility:** Standard wallets (EOAs) have a single point of failure; if your key is stolen, your funds are gone instantly.
- **The Vision:** To build a programmable vault that "thinks" before it executes and "verifies" without seeing.

Slide 3: Existing Solutions

- **Current State:**
 1. **Metamask/EOAs:** Simple but "dumb" wallets. No logic, no recovery, no multi-factor protection.
 2. **Centralized KYC:** High privacy risk. Users must trust a third party with their data.
 3. **Standard Multi-sig:** Requires multiple humans to be online to sign, which is slow and inefficient for individual users.

Slide 4: Proposed Solution

- **The LockedFI Innovation:**
 1. **ZK-Income Attestation:** Uses the DKIM signatures already present in emails to prove financial status without revealing the email content.
 2. **AI Guardian (Llama-3):** An autonomous security agent that analyzes transaction risk in real-time.
 3. **ERC-4337 Smart Vaults:** Programmable "Smart Accounts" that enforce the rule: *No high-value withdrawal without AI approval.*

Slide 5: Detailed Architecture

- **Layer 1 (Identity):** Circom circuits generate a ZK-Proof of income.
- **Layer 2 (Intelligence):** Groq-powered Llama-3 processes off-chain context (history, amount, badge status).
- **Layer 3 (Execution):** Solidity Smart Vault on Arbitrum/Anvil verifies signatures and releases funds.
- *Visual Hint: Draw a 3-tier pyramid: Identity (Bottom), Intelligence (Middle), Execution (Top).*

Slide 6: Modules (The List)

1. **ZK-Verifier Module (Solidity):** Located on-chain. *Why:* To verify mathematical proofs without seeing the data.
2. **AI Co-signer Module (Python/Groq):** Located off-chain. *Why:* To provide a second "intelligent" layer of security that can detect unusual behavior.
3. **Account Abstraction Module (ERC-4337):** Located in the Smart Vault. *Why:* To allow the vault to have custom logic (like the `validateUserOp` function) that standard wallets don't have.

- 4. **The Bundler Bridge (Next.js API):** Located in the frontend. *Why:* To glue the human signature and the AI signature together into one payload.

Slide 7: Project Demonstration

- **Step 1:** Connect Brave Wallet to the LockedFI Dashboard.
 - **Step 2:** Display the "Green Shield" (ZK-Income Verified Badge).
 - **Step 3:** Initiate a 0.05 ETH withdrawal.
 - **Step 4:** User signs in Brave; AI Guardian analyzes and co-signs automatically.
 - **Step 5:** Smart Vault verifies the dual-signature and releases funds.
-

Presentation Slides (Expanded Content)

Slide 1: Abstract

- **The Core Concept:** LockedFI is a decentralized "Smart Vault" architecture that integrates Zero-Knowledge (ZK) cryptography with Artificial Intelligence to solve the "Privacy-Security Paradox" in DeFi.
- **Problem Solved:** Traditional DeFi requires users to either sacrifice privacy (by uploading financial documents) or security (by relying on a single private key).
- **The Innovation:** We utilize ZK-Email/DKIM proofs to attest to a user's income threshold without revealing sensitive metadata. This is coupled with an autonomous AI Guardian (Llama-3) that acts as a real-time co-signer.
- **Outcome:** A programmable safe that enforces a 2-of-2 security model (Owner + AI) for high-risk transactions, ensuring that even if a private key is stolen, the assets remain protected by an intelligent off-chain layer.

Slide 2: Introduction

- **The Evolution of Wallets:** We are moving from "Static Wallets" (like MetaMask) to "Intelligent Accounts." Current wallets are "dumb"—they follow any signed instruction, even if it is a clear hack.
- **Privacy in the Digital Age:** Financial status is a requirement for many services (lending, premium access), but putting that data on a public ledger is a permanent privacy violation.
- **Project Mission:** To develop a system where:
 1. **Privacy is Default:** Users prove they are qualified for a service using ZK-Proofs.
 2. **Security is Adaptive:** The system analyzes the *context* of a transaction (amount, history, badge status) before allowing execution.
 3. **Governance is Autonomous:** Removing the need for slow, human-managed multi-signs in favor of a fast AI co-signer.

Slide 3: Existing Solutions & Gap Analysis

- **Externally Owned Accounts (EOAs):**
 - *Flaw:* Single point of failure. If your private key is phished, the vault is drained in seconds.
 - *Gap:* No internal logic or risk assessment.
- **Centralized KYC/Income Verification:**
 - *Flaw:* Requires users to hand over passports and bank statements to third-party databases, which are frequently hacked.
 - *Gap:* No user sovereignty over data.
- **Traditional Multi-Signature Wallets (Safe/Gnosis):**
 - *Flaw:* Requires multiple humans to sign. This causes high friction and delays for individual users.
 - *Gap:* Not scalable for instant, day-to-day high-security transactions.

Slide 4: Proposed Solution: The LockedFI Framework

- **Pillar 1: ZK-Income Attestation:** Using the **Groth16** proving system, we allow users to generate a proof that their income exceeds a threshold based on hidden email data. The blockchain verifies the "Badge" but never sees the email.
- **Pillar 2: AI Guardian Co-signer:** We implement an off-chain AI agent powered by **Llama-3 (Groq)**. This agent receives transaction details, performs a risk-reward analysis, and provides a second cryptographic signature only for legitimate requests.
- **Pillar 3: ERC-4337 Smart Vaults:** We move away from basic wallets to **Programmable Smart Contracts**. This allows us to hard-code security rules: "If Amount > 0.1 ETH, require ZK-Badge AND AI-Signature."

Slide 5: Detailed Architecture & System Design

- **The Frontend Layer (Next.js):** Acts as the user's dashboard. It coordinates wallet connection, ZK-proof submission, and requests the AI Guardian's approval via a secure API bridge.
- **The Intelligence Layer (Python/Groq):** An off-chain service that holds the Guardian's private key. It uses a high-speed inference engine to decide whether to sign a transaction hash based on the provided context.
- **The Cryptographic Layer (Circom/SnarkJS):** Handles the generation of the ZK-Verifier contract. It translates mathematical constraints into a Solidity "lock" that the Smart Vault uses.
- **The Execution Layer (Solidity/Arbitrum):** The on-chain Smart Vault. It contains the `validateUserOp` function which checks the "Super-Signature" (Owner Signature + AI Signature) before releasing funds.

Slide 6: System Modules (The "Why" and "Where")

1. **ZK-Verifier Module (On-chain):** A generated Solidity contract that checks the validity of income proofs. *Why:* To ensure the user is "Qualified" without needing an Oracle or a human reviewer.
2. **AI Risk-Analysis Module (Off-chain):** A Python service using the Groq SDK. *Why:* To provide "Intelligent Security" that can distinguish between a user's normal behavior and a potential hack.
3. **Account Abstraction Module (On-chain):** Custom logic within `SmartVault.sol`. *Why:* To enable "Dual-Key" protection. It allows the vault to reject transactions that don't have the AI's approval.
4. **Bundler Bridge API (Next.js API):** A secure server-side route. *Why:* To prevent exposing the AI Guardian's private key to the browser, keeping the "Security Guard" in a protected environment.

Slide 7: Project Demonstration & Workflow

- **Phase 1: Connection & Sync:** The user connects their Brave Wallet. The dashboard syncs with the Smart Vault on the Anvil local blockchain.
 - **Phase 2: Privacy Attestation:** The user displays their ZK-Income Badge. This confirms the on-chain `hasHighIncomeBadge` state is true, unlocking higher withdrawal limits.
 - **Phase 3: The Request:** The user attempts to withdraw 0.05 ETH. The frontend generates a unique transaction hash.
 - **Phase 4: The Dual-Signature:**
 1. The User signs the hash in Brave.
 2. The AI Guardian analyzes the request and signs the hash.
 3. The two signatures are bundled into a 130-byte payload.
 - **Phase 5: Execution:** The Smart Vault verifies both signatures and the ZK-Badge, then successfully releases the funds.
-